

Reliable Text-to-SQL Backend Project Plan — Dual RAG Update

Updated roadmap with dual retrieval-augmented generation: one retriever for few-shot examples and one retriever for schema context, plus dedicated retrieval evaluation.

Project Vision

Build a reliable text-to-SQL backend service that accepts natural-language questions, retrieves the most relevant schema context and few-shot demonstrations, generates SQL through a structured multi-stage pipeline, validates and executes it in a safe read-only environment, returns the answer plus trace metadata, and continuously improves through offline and online evaluation.

What Changed in This Revision

- Dual RAG is now a first-class design element: one retrieval path for **few-shot examples** from the training set and another for **schema retrieval**.
- Retrieval quality is now an explicit evaluation surface rather than hidden prompt plumbing.
- The structured pipeline is no longer only “question → one-shot SQL”; it includes retrieval, narrowing, draft generation, refinement, and targeted repair.
- This creates a cleaner way to compare bounded structured context versus more open agent behavior and to measure where additional structure helps or hurts.

Core Design Principle

Treat this as a **backend system first** and an **agent system second**. Retrieval, validation, execution, and evaluation should be independently observable and testable. The agent layer should be bounded by structured retrieval outputs whenever possible so that you can study the impact of better context rather than only the raw behavior of an unconstrained agent.

High-Level System Variants to Compare

- **Variant A — One-shot baseline:** direct question + full schema prompt to SQL generation.
- **Variant B — Dual-RAG structured pipeline:** few-shot retrieval + schema retrieval + staged SQL generation.
- **Variant C — Agentic/tool-using workflow:** the model can call tools to inspect schema and possibly retrieve examples dynamically.
- **Variant D — Hybrid:** structured dual-RAG context is given up front, then the agent may use tools only when uncertainty remains high.

Why Dual RAG Matters

- It gives you an additional learning objective beyond SQL generation: how to build, index, retrieve, rank, and evaluate context sources.
- It creates natural ablation studies: full schema vs narrowed schema, no examples vs retrieved examples, static structured pipeline vs agentic retrieval.
- It helps bound the agent by pre-computing structured evidence rather than relying only on open-ended tool exploration.
- It gives you two new evaluation axes: schema retrieval quality and few-shot example retrieval quality.

Recommended End-to-End Pipeline (Updated)

```
User question
-> request classification
-> schema retriever
-> few-shot example retriever
-> prompt assembly for stage-1 generation
-> draft SQL (PreSQL / candidate SQL)
-> parse draft for mentioned tables / columns / entities
-> schema narrowing / schema linking refinement
-> stage-2 final SQL generation
-> SQL validation
-> SQL execution
-> answer formatting
-> optional critique / repair loop
-> optional human review
```

Pipeline Stages and Rationale

- **Request classification:** decide whether the question is in scope, ambiguous, or likely to require escalation.
- **Schema retrieval:** retrieve the most relevant tables, columns, relationships, descriptions, and optionally sample values or metadata.
- **Few-shot retrieval:** retrieve similar training examples, ideally including natural-language question, SQL, difficulty tags, and query skeleton features.
- **Stage-1 generation:** produce a draft SQL using retrieved schema context and demonstrations.
- **Schema refinement:** parse the draft SQL to infer likely tables/columns and compact the schema prompt further.
- **Stage-2 generation:** produce the final SQL with a tighter, more focused schema context.
- **Validation and execution:** enforce read-only policy, validate syntax/constraints, and execute under time/row budgets.
- **Repair:** only if needed, critique the failure and regenerate with bounded retries.

Technology Roles

- **LangChain:** model abstraction, prompt plumbing, retrieval chains, structured outputs, and embedding interfaces.
- **LangGraph:** orchestration for the multi-stage workflow, repair routing, pause/resume, and bounded tool use.
- **LangSmith:** datasets, evaluators, experiments, regression tracking, retrieval evaluation, and online/offline analysis.
- **Vector store + embeddings:** indexes for schema chunks and few-shot training examples.
- **FastAPI / SQLAlchemy / Redis / Queue Worker:** service layer, metadata persistence, caching, async jobs, and batch evaluation execution.

Updated Build Order

- Core backend + SQL execution layer
- Schema introspection and metadata representation
- Dual retrieval layer: schema index + few-shot example index
- Deterministic structured pipeline using dual-RAG context
- Offline evaluation harness including retrieval metrics
- LLM integration with structured outputs
- LangGraph orchestration
- Repair / self-critique loop
- Guardrails
- Human-in-the-loop review

- Async jobs and regression pipelines
- Agentic comparison variants and online evaluation

Module 0 — Foundation and Constraints

Goal: Define project boundaries, data sources, retrieval policy, query policy, response contract, and job model before writing any agent logic.

Responsibilities

- Choose the initial database target: start with SQLite locally and optionally add PostgreSQL later.
- Restrict the service to read-only SELECT queries.
- Define the response contract: answer, generated SQL, retrieved schema context summary, retrieved examples summary, execution metadata, confidence, flags, and trace ID.
- Decide which flows are synchronous versus asynchronous jobs.
- Define whether retrieval uses only Spider training data, handcrafted examples, or both.

Expected Obstacles

- Scope creep from trying to support too many retrieval sources or databases too early.
- Not clearly separating retrieval quality from generation quality.
- Building an “agent demo” before you define what success means.

Learning Outcomes

- Boundary setting for AI systems
- API contract thinking
- Evaluation-aware system planning
- How to turn a fuzzy AI idea into concrete interfaces and measurable objectives

Module 1 — Schema Introspection and Metadata Representation

Goal: Build the database understanding layer that will later feed both schema retrieval and SQL generation.

Responsibilities

- Connect to the database and inspect tables, columns, foreign keys, relationships, and constraints.
- Create a normalized schema representation that can be chunked and indexed for retrieval.
- Optionally include descriptions, aliases, foreign-key paths, and sample values to enrich retrieval documents.
- Cache schema metadata for performance and repeatability.

Expected Obstacles

- Finding the right chunk granularity for retrieval: whole table documents vs column-level fragments.
- Choosing how much metadata to include without blowing up token budgets.
- Representing joins and relationships in a retrieval-friendly format.

Learning Outcomes

- Metadata modeling
- Retrieval-oriented document design
- Practical schema engineering for LLM systems

Module 2 — Dual RAG Layer

Goal: Implement two complementary retrievers: one for schema context and one for few-shot demonstrations.

Responsibilities

- Build a **schema retriever** that indexes schema documents and returns the most relevant tables, columns, relationships, and metadata for a question.
- Build a **few-shot retriever** that indexes training examples and returns the most relevant question-SQL pairs.
- Design feature-rich example documents: question, SQL, simplified SQL skeleton, tables used, query type, difficulty, and notes.
- Support multiple retrieval strategies: embedding-based similarity, lexical filters, metadata filters, or hybrid ranking.
- Log retrieved items so retrieval can be evaluated independently of generation.

Expected Obstacles

- Examples that look semantically similar but have misleading SQL structure.
- Schema retrieval returning too many irrelevant columns or missing key join paths.
- Prompt length explosion when combining retrieved examples and retrieved schema context.
- Training-set contamination concerns if the project later expands beyond Spider-style evaluation.

Learning Outcomes

- Embedding and vector search basics
- Hybrid retrieval design
- Context ranking and curation
- Why retrieval is not just a plumbing detail but a core system capability

Module 3 — SQL Validator and Safe Executor

Goal: Build the execution layer that remains fully usable without any LLM dependency.

Responsibilities

- Validate SQL against strict read-only policies.
- Apply timeouts, row caps, and execution budgets.
- Normalize result sets for downstream logic and evaluation.
- Store structured execution outcomes and error classes.

Expected Obstacles

- Designing validation rules strict enough to block unsafe SQL without over-rejecting valid queries.
- Handling execution failures cleanly and consistently.
- Normalizing result sets across different query shapes.

Learning Outcomes

- Practical SQL safety engineering
- Execution budgeting
- Clean exception and error-class design

Module 4 — Deterministic Structured Baseline Pipeline

Goal: Create a serious baseline that already uses structured retrieval before any graph-based orchestration.

Pipeline

- Receive the user question.
- Retrieve schema context.
- Retrieve few-shot examples.

- Assemble a structured prompt.
- Generate draft SQL.
- Optionally narrow the schema using the draft.
- Generate final SQL.
- Validate and execute.
- Format the response.

Expected Obstacles

- Prompt assembly becoming too large or noisy.
- Retrieved examples steering the model toward the wrong SQL pattern.
- Schema narrowing missing a needed join path.
- Semantically wrong but syntactically valid SQL.

Learning Outcomes

- Prompt-to-structure design
- Retrieval-aware SQL generation
- How to compare no-retrieval vs retrieved-context pipelines

Module 5 — Offline Evaluation Harness (Expanded for Retrieval)

Goal: Measure not only final SQL quality but also whether retrieval is actually doing useful work.

Responsibilities

- Create a small manually curated dataset first, then add a Spider subset.
- Store per-example metadata: difficulty, number of joins, query type, tables used, SQL skeleton, ambiguity level.
- Log all retrieved schema items and retrieved few-shot examples for every run.
- Implement evaluators for final SQL quality and separate evaluators for retrieval quality.

Retrieval Evaluation Metrics

- **Schema retrieval recall:** how many gold-relevant tables/columns were retrieved?
- **Schema retrieval precision / noise ratio:** how much irrelevant schema was added?
- **Join-path coverage:** did retrieval include the relationships needed for the gold query?
- **Few-shot usefulness:** did retrieved examples share the right SQL pattern, tables, or skeleton features?
- **Retriever contribution ablations:** compare no retrieval vs schema-only vs examples-only vs dual-RAG.
- **Prompt budget efficiency:** what accuracy gain do you get per added token of retrieved context?

Generation/Execution Metrics

- Execution success
- Result equivalence
- Latency
- Retry count
- SQL safety
- Cost per query

Expected Obstacles

- Gold relevance labels for retrieval can be non-trivial to define.
- An example may look “similar” semantically but be structurally unhelpful.
- Retrieval metrics and generation metrics may disagree, which requires careful diagnosis.

Learning Outcomes

- How to evaluate retrievers in an application-specific way
- Ablation design
- Regression testing mindset
- The difference between context quality and final answer quality

Module 6 — Observability, Tracing, and Failure Taxonomy

Goal: Make retrieval and generation equally debuggable before introducing loops and retries.

Responsibilities

- Log prompt version, model version, schema index version, few-shot index version, retrieved item IDs, chosen tables, generated SQL, validation results, and execution results.
- Track latency per retrieval step and per generation step.
- Define a failure taxonomy that includes retrieval failures as first-class categories.

Suggested Failure Categories

- Schema recall failure: needed table/column not retrieved.
- Schema noise failure: too much irrelevant context overwhelms the model.
- Few-shot misguidance: retrieved examples bias the model toward the wrong SQL pattern.
- Hallucinated schema reference.
- Wrong join path.
- Timeout, empty result, or fabricated answer.

Learning Outcomes

- Production debugging discipline
- Failure categorization across retrieval and generation
- How observability and evaluation complement each other

Module 7 — LangGraph Orchestration

Goal: Turn the deterministic structured pipeline into an inspectable stateful workflow.

Suggested Graph

- `classify_request`
- `retrieve_schema_context`
- `retrieve_few_shot_examples`
- `generate_draft_sql`
- `refine_schema_context`
- `generate_final_sql`
- `validate_sql`
- `execute_sql`
- `format_answer`

Conditional Edges

- If retrieval confidence is too low, escalate to broader retrieval or human review.
- If SQL validation fails, route to critique or repair logic.
- If execution fails, route to critique or repair logic.
- If execution succeeds, format the final answer.

Expected Obstacles

- Over-engineering the graph too early.
- Unclear state schema for retrieved context.
- Replay or idempotency issues once persistence is introduced.

Learning Outcomes

- State-machine thinking
- Workflow decomposition
- How to orchestrate retrieval-heavy structured pipelines

Module 8 — Repair / Self-Critique Loop

Goal: Add bounded recovery not only for SQL but also for retrieval mistakes.

Responsibilities

- Introduce critique_sql and repair_sql nodes.
- Introduce retrieval critique logic: determine whether the failure came from bad schema context, bad examples, or bad generation.
- Allow bounded fallback strategies such as broader schema retrieval, alternative few-shot retrieval, or example suppression.
- Re-run generation with a strict retry budget.

Expected Obstacles

- Repair loops that simply add more context and make prompts worse.
- Difficulty separating retrieval fault from generation fault.
- Retry cost explosion in tokens and latency.

Learning Outcomes

- Bounded autonomy
- Targeted retries instead of blind retries
- How to reason about failures in multi-stage pipelines

Module 9 — Guardrails and Bounded Agent Behavior

Goal: Use structured retrieval outputs to control and study agent behavior rather than allowing unconstrained exploration by default.

Guardrail Types

- Input guardrails: classify whether the request is a valid data question and whether it is too ambiguous or unsafe.
- Retrieval guardrails: cap schema context size, cap number of few-shot examples, and reject low-confidence retrieval configurations.
- SQL guardrails: enforce SELECT-only behavior, row limits, timeouts, and table sensitivity rules.
- Output guardrails: prevent fabricated answers if execution failed and require uncertainty disclosure when fallback paths were used.

Why This Matters

- It lets you compare structured-context performance against agentic tool exploration.
- It helps measure whether adding more structure actually improves behavior or merely increases prompt cost.
- It creates a safer experimental space for studying where agent freedom is helpful.

Module 10 — Human-in-the-Loop, Async Jobs, and Regression Gates

Goal: Bring the system closer to a production workflow and enable larger retrieval/generation experiments.

Use Cases

- Long-running evaluations and ablation studies
- Batch retrieval benchmarking
- Scheduled regressions
- Slow multi-step agent workflows
- Human review for low-confidence retrieval or repeated repair failure

Example Regression Gates

- Execution success rate must not drop by more than a chosen threshold.

- Schema retrieval recall must remain above a chosen threshold on the curated eval set.
- Few-shot retrieval usefulness score must not regress.
- Unsafe SQL attempt rate must not increase.
- Latency and prompt cost must stay within budget.

Learning Outcomes

- Release criteria for AI systems
- Continuous evaluation
- Experiment-driven development across retrieval and generation layers

Suggested LangGraph State (Updated for Dual RAG)

- request_id — unique request identifier for tracing and persistence
- thread_id — workflow/thread identifier for checkpointed runs
- user_question — raw natural-language input
- db_id — target database identifier
- retrieved_schema_items — tables, columns, relationships, and metadata returned by schema retrieval
- retrieved_examples — question-SQL demonstrations returned by few-shot retrieval
- schema_retrieval_scores — ranking scores and retrieval metadata for schema context
- example_retrieval_scores — ranking scores and retrieval metadata for few-shot examples
- draft_sql — first-stage candidate SQL
- candidate_sql — final SQL candidate being validated or executed
- validation_result — policy and syntax validation output
- execution_result — raw and normalized SQL execution output
- error_category — classified failure category
- retry_count — number of repair attempts already used
- final_answer — user-facing answer after successful execution
- decision_flags — policy signals such as human review required, retrieval low confidence, high risk, timeout risk
- trace_metadata — prompt version, model version, index version, latency, cost, and similar metadata

Graph Evolution

Version 1 – One-shot baseline

question -> full_schema_prompt -> generate_sql -> validate_sql -> execute_sql -> format_answer

Version 2 – Dual-RAG structured baseline

question -> retrieve_schema -> retrieve_examples -> generate_draft_sql -> refine_schema -> generate_final_sql -> validate

Version 3 – Retrieval-aware repair path

... -> if validation_error or execution_error: critique_failure -> decide(retrieval_fault?, generation_fault?) -> broaden

Version 4 – Policy layers

input_guardrail -> retrieve_schema -> retrieve_examples -> retrieval_guardrail -> generate_sql -> sql_guardrail -> execute

Version 5 – Hybrid agent

precomputed_dual_rag_context -> agent_with_tools -> optional tool use only when uncertainty remains high

Priority Evaluation Metrics

Tier 1 — Retrieval metrics

- schema retrieval recall
- schema retrieval precision / noise ratio
- join-path coverage
- few-shot structural usefulness
- retrieval latency
- retrieval token budget efficiency

Tier 2 — Generation / execution metrics

- execution_success
- result_equivalence

- sql_safety
- latency
- retry_count
- cost_per_query

Tier 3 — Comparative / ablation metrics

- no retrieval vs schema-only vs examples-only vs dual-RAG
- structured pipeline vs agentic pipeline
- prompt cost versus accuracy gain
- human-escalation rate and repair success rate

Milestones

Milestone A — Backend-first MVP

- API
- schema introspection
- SQL validator and executor
- schema retriever
- few-shot retriever
- deterministic dual-RAG baseline
- basic tracing
- small offline evaluation dataset

Milestone B — Reliable structured workflow

- retrieval-aware evaluation metrics
- LangGraph state machine
- draft SQL + refinement stage
- retrieval-aware repair loop
- guardrails
- async evaluation jobs

Milestone C — Controlled agent experiments

- hybrid agent using structured retrieval context
- bounded tool access
- online evaluation capture
- nightly regression
- failure dataset mining
- comparative experiments against one-shot and tool-using baselines

Suggested Repository Structure (with inline role comments)

```
text2sql_service/
  app/
    api/
      routes_query.py
      routes_jobs.py
      routes_eval.py
      routes_admin.py
    core/
      config.py
      logging.py
      tracing.py
      errors.py
      security.py
    db/
      engine.py
      metadata_store.py
      schema_introspection.py
      sql_validator.py
      sql_executor.py
      result_normalizer.py
    retrieval/
      embeddings.py
      vector_store.py
      schema_docs.py
      example_docs.py
      schema_retriever.py
      example_retriever.py
      rerankers.py
      retrieval_metrics.py
    llm/
      prompts.py
      models.py
      structured_types.py
      prompt_assembly.py
      sql_generator.py
      sql_reviewer.py
      answer_generator.py
    orchestration/
      state.py
      graph.py
      nodes/
        classify_request.py
        retrieve_schema_context.py
        retrieve_examples.py
        generate_draft_sql.py
        refine_schema_context.py
        generate_final_sql.py
        validate_sql.py
        execute_sql.py
        critique_failure.py
        repair_sql.py
        adjust_retrieval.py
        format_answer.py
        request_human_review.py
      policies/
        input_guardrails.py
        retrieval_guardrails.py
        sql_guardrails.py
        output_guardrails.py
    workers/
      queue.py
      tasks_query.py
      tasks_eval.py
    eval/
      datasets/
        offline_runner.py
        online_capture.py
        retrieval_eval.py
        ablations.py
        evaluators/
          exact_match.py
          execution_success.py
          result_equivalence.py
          sql_safety.py

# Root repository for the backend service, retrieval system, orchestration
# Main application package containing the runnable service code
# HTTP layer exposing query, job, evaluation, and admin endpoints
# Request/response handlers for end-user text-to-SQL queries
# Endpoints for submitting, inspecting, and managing async jobs
# Endpoints for launching and reviewing evaluation runs
# Operational/admin endpoints for configuration, debugging, or review tool
# Shared cross-cutting infrastructure used across the whole service
# Centralized settings, environment variables, and feature flags
# Structured logging configuration and log helpers
# Trace and correlation-ID setup for request and workflow observability
# Shared exception types and service-level error handling utilities
# Global security helpers such as auth, secrets, and policy enforcement hooks
# Database-facing layer for metadata, validation, and execution
# Database connection/session management and engine initialization
# Storage and caching of normalized schema metadata snapshots
# Logic to inspect tables, columns, relationships, and schema features
# Read-only policy checks, syntax checks, and SQL safety validation
# Safe SQL execution with timeouts, row caps, and execution controls
# Consistent formatting of query results for downstream logic and APIs
# Dual-RAG components for schema retrieval and few-shot example retrieval
# Embedding model setup and shared embedding utilities
# Vector index abstraction and persistence for retrieval documents
# Conversion of schema metadata into retrieval-friendly documents/chunks
# Conversion of training examples into retrieval-friendly few-shot documents
# Retrieval pipeline that ranks relevant schema context for a question
# Retrieval pipeline that ranks relevant few-shot demonstrations
# Optional reranking logic for hybrid search or metadata-aware ranking
# Shared helpers for measuring retrieval recall, noise, and usefulness
# Model-facing code for prompt construction and typed outputs
# Prompt templates and prompt versioning for generation, critique, and feedback
# Model client initialization and provider abstraction
# Pydantic or typed schemas for model outputs and node contracts
# Logic that combines retrieved schema and examples into bounded prompts
# Stage-1 and stage-2 SQL generation logic using retrieved context
# Critique/review logic for failed or suspicious SQL candidates
# Natural-language answer formatting grounded in execution results
# LangGraph workflow definitions, state, nodes, and routing policies
# Workflow state schema shared across nodes and persisted in checkpoints
# Main LangGraph assembly: nodes, edges, conditional routing, compilation
# Individual workflow steps implemented as modular node functions
# Detects question type, ambiguity, and whether the request is in scope
# Retrieves the most relevant schema context and stores ranked results
# Retrieves the most relevant few-shot demonstrations and stores ranked results
# Produces a draft SQL using the initial retrieved context
# Narrows or restructures schema context using the draft SQL
# Produces the final SQL candidate using refined context
# Applies SQL policy and syntax validation before execution
# Runs validated SQL and stores execution outcomes in state
# Diagnoses whether failure came from retrieval, generation, or execution
# Produces revised SQL candidates after critique or validation failure
# Broadens, filters, or re-ranks retrieval when context quality is suspected
# Converts execution output into the final user-facing answer
# Pauses workflow for human approval, clarification, or intervention
# Guardrail and routing rules used across the graph
# Rules for rejecting or escalating unsafe / irrelevant user inputs
# Rules for capping retrieval size, context noise, and low-confidence retrieval
# Rules for enforcing read-only SQL, table sensitivity, and budgets
# Rules for ensuring the final answer is grounded and policy-compliant
# Background-job infrastructure for async execution and batch processing
# Queue client and worker setup (Celery, RQ, Dramatiq, etc.)
# Background tasks for long-running query workflows
# Background tasks for dataset evaluation and regression runs
# Evaluation datasets, evaluators, experiment runners, and reports
# Stored curated examples, benchmark subsets, and production-derived cases
# Code to run offline experiments on fixed datasets and ablation settings
# Logic to capture and promote production traces into evaluation candidates
# Evaluation runner focused on schema retrieval and few-shot retrieval quality
# Utilities to compare no-retrieval, schema-only, examples-only, and dual-RAG
# Individual metric implementations for scoring runs
# Optional exact SQL-string comparison for debugging or analysis
# Metric for whether the generated SQL executed successfully
# Metric comparing result sets or normalized answers to reference outputs
# Metric tracking unsafe SQL attempts or policy violations
```

■	■	■■■ latency.py	# Metric for latency budgets and performance regressions
■	■	■■■ schema_recall.py	# Metric measuring whether needed tables/columns were retrieved
■	■	■■■ schema_noise.py	# Metric measuring irrelevant context included by schema retrieval
■	■	■■■ fewshot_usefulness.py	# Metric estimating whether retrieved demonstrations were structurally helpful
■	■	■■■ tool_efficiency.py	# Metric for graph/tool usage efficiency and unnecessary retries
■	■	■■■ llm_judge.py	# Optional model-based evaluator for faithfulness or answer quality
■	■	■■■ reporting.py	# Aggregation and export of experiment summaries, charts, and findings
■■■ tests/			# Unit, integration, retrieval, and regression tests for every layer of the system
■■■ docker/			# Container and local environment definitions for reproducible development
■■■ scripts/			# Utility scripts for setup, dataset prep, indexing, benchmarking, and maintenance
■■■ docs/			# Project documentation, architecture notes, and design records
■	■■■	architecture.md	# High-level architecture, data flow, and module interaction notes
■	■■■	retrieval.md	# Design of schema retrieval and few-shot retrieval, including indexing details
■	■■■	evaluation.md	# Evaluation philosophy, metrics, datasets, ablations, and regression procedures
■	■■■	failure_taxonomy.md	# Canonical list of retrieval, generation, and execution failure modes
■■■		README.md	# Entry point explaining the project, setup, usage, and design rationale

Summary Recommendations

- Make dual-RAG the default serious baseline, not just an optional experiment.
- Treat retrieval quality as a first-class metric and log every retrieved item.
- Use structured retrieval outputs to bound agent behavior and to study where more open tool use actually helps.
- Prioritize ablations: no retrieval vs schema-only vs examples-only vs dual-RAG vs hybrid agent.
- Keep repair and escalation bounded, observable, and easy to explain.